

Chapter Seven

Multithreading Programming

Thread

- Thread: single sequential flow of control within a program
- Single-threaded program can handle one task at any time
- Multitasking allows single processor to run several concurrent threads
- Most modern operating systems support multitasking

Advantages of Multithreading

- Reactive systems – constantly monitoring
- More responsive to user input – GUI application can interrupt a time-consuming task
- Server can handle multiple clients simultaneously
- Can take advantage of parallel processing
- Different processes do not share memory space.
- A thread can execute concurrently with other threads within a single process
- All threads managed by the JVM share memory space and can communicate with each other

Threads Concept

Multiple
threads on
multiple
CPUs



Multiple
threads
sharing a
single CPU

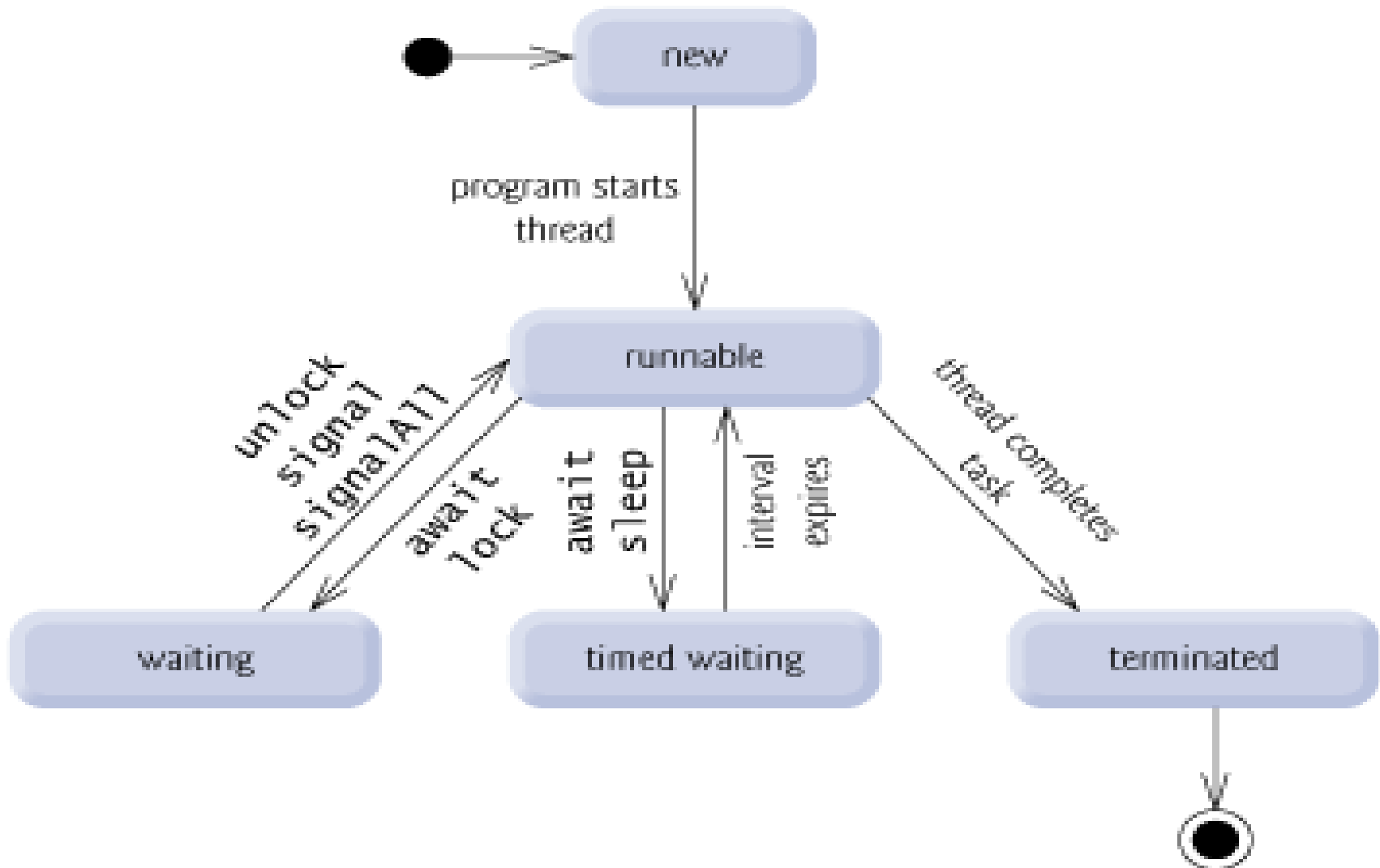


Threads in Java

Creating threads in Java:

- Extend **java.lang.Thread** class
 - run() method must be overridden (similar to main method of sequential program)
 - run() is called when execution of the thread begins
 - A thread terminates when run() returns
 - start() method invokes run()
 - Calling run() does not create a new thread
- Implement **java.lang.Runnable** interface
 - If already inheriting another class (i.e., JApplet)
 - Single method: public void run()
 - Thread class implements Runnable.

Thread States



Thread termination

- A thread becomes Not Runnable when one of these events occurs:
 - Its sleep method is invoked.
 - The thread calls the wait method to wait for a specific condition to be satisfied.
 - The thread is blocking on I/O.

Creating Tasks and Threads

java.lang.Runnable ← --- TaskClass

```
// Custom task class
public class TaskClass implements Runnable {
    ...
    public TaskClass(...) {
        ...
    }

    // Implement the run method in Runnable
    public void run() {
        // Tell system how to run custom thread
        ...
    }
    ...
}
```

```
// Client class
public class Client {
    ...
    public void someMethod() {
        ...
        // Create an instance of TaskClass
        TaskClass task = new TaskClass(...);

        // Create a thread
        Thread thread = new Thread(task);

        // Start a thread
        thread.start();
        ...
    }
    ...
}
```

Exercise: Using the Runnable Interface to Create and Launch Threads

- Objective: Create and run three threads:
 - The first thread prints the letter *a* 100 times.
 - The second thread prints the letter *b* 100 times.
 - The third thread prints the integers 1 through 100.

The Thread Class

«interface»
java.lang.Runnable



java.lang.Thread

+Thread()
+Thread(task: Runnable)
+start(): void
+isAlive(): boolean
+setPriority(p: int): void
+join(): void
+sleep(millis: long): void
+yield(): void
+interrupt(): void

- Creates a default thread.
- Creates a thread for a specified task.
- Starts the thread that causes the run() method to be invoked by the JVM.
- Tests whether the thread is currently running.
- Sets priority p (ranging from 1 to 10) for this thread.
- Waits for this thread to finish.
- Puts the runnable object to sleep for a specified time in millisecond.
- Causes this thread to temporarily pause and allow other threads to execute.
- Interrupts this thread.

The Static `yield()` Method

You can use the `yield()` method to temporarily release time for other threads. For example,

```
public void run() {  
    for (int i = 1; i <= lastNum; i++) {  
        System.out.print(" " + i);  
        Thread.yield();  
    }  
}
```

Every time a number is printed, the print thread is yielded. So, the numbers are printed after the characters.

The Static sleep(milliseconds) Method

The `sleep(long mills)` method puts the thread to sleep for the specified time in milliseconds.

For example,

```
public void run() {  
    for (int i = 1; i <= lastNum; i++) {  
        System.out.print(" " + i);  
        try {  
            if (i >= 50) Thread.sleep(1);  
        }  
        catch (InterruptedException ex)  
        { }  
    }  
}
```

Every time a number (≥ 50) is printed, the print100 thread is put to sleep for 1 millisecond.

Example: GreetingRunnable.java

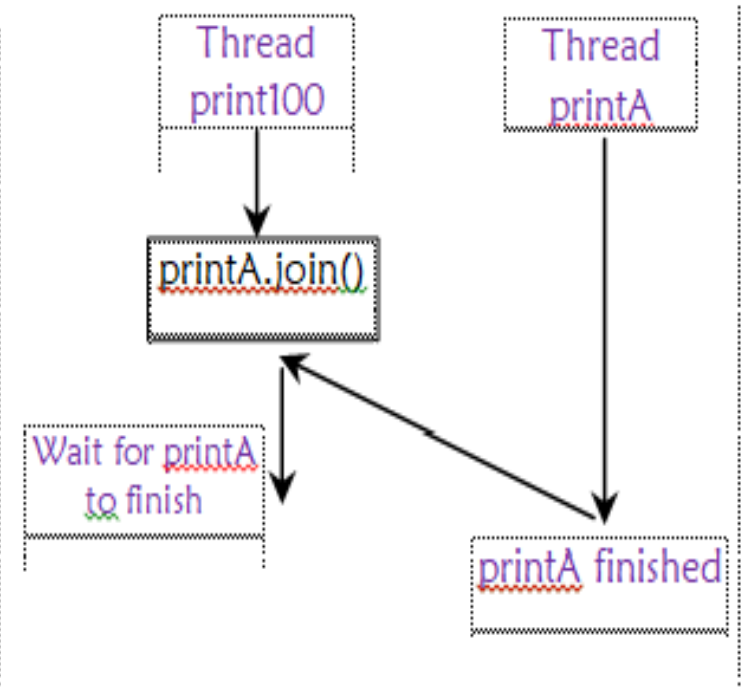
```
import java.util.Date;
public class GreetingRunnable implements Runnable {
    private static final int REPETITIONS = 10;
    private static final int DELAY = 1000;
    private String greeting;
    public GreetingRunnable(String aGreeting){
        greeting = aGreeting;
    }
    public void run(){
        try{
            for (int i = 1; i <= REPETITIONS; i++){
                Date now = new Date();
                System.out.println(now+" "+ greeting);
                Thread.sleep(DELAY);
            }
        }catch (InterruptedException exception) {}
    }
}
```

The join() Method

You can use the join() method to force one thread to wait for another thread to finish.

For example

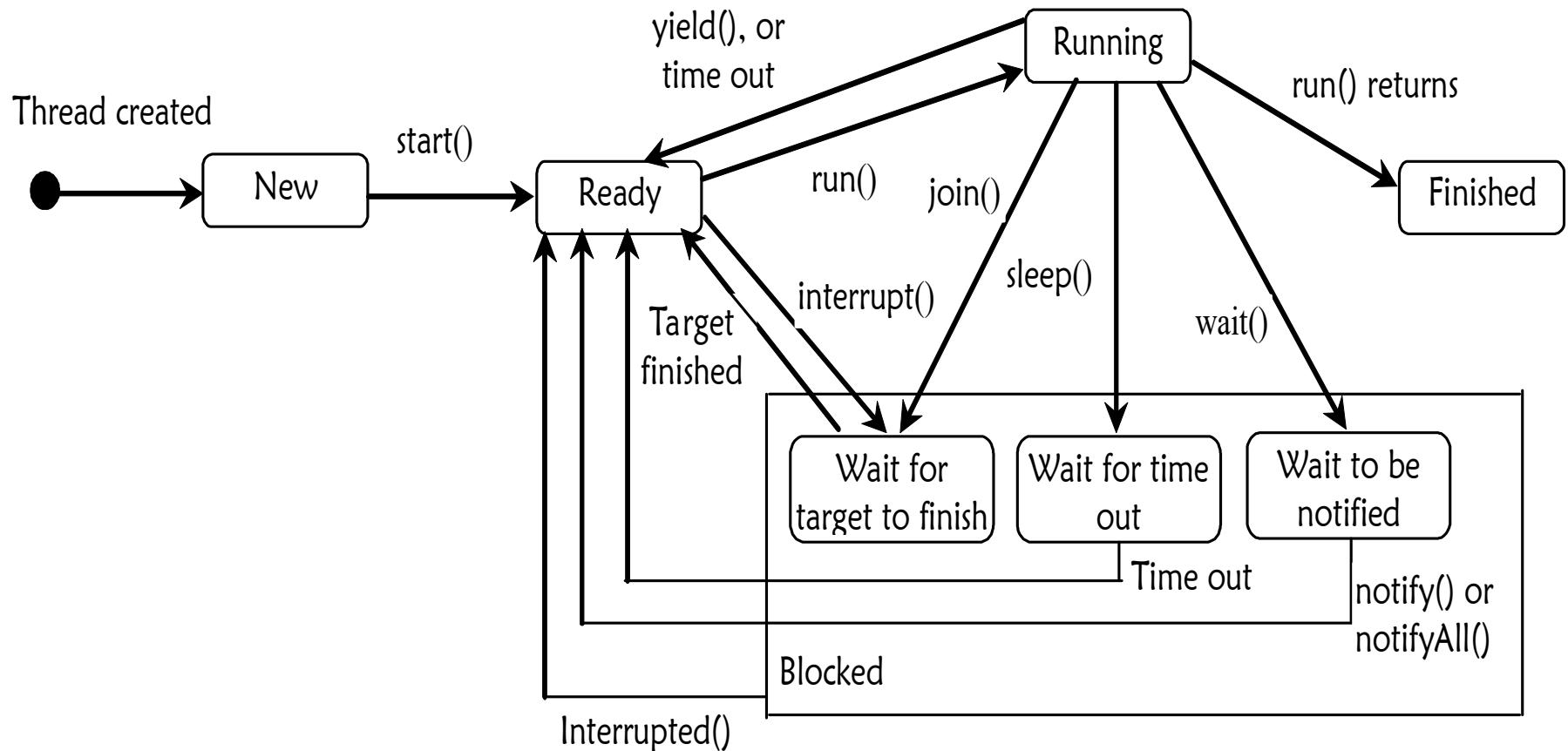
```
public void run() {  
    Thread thread4 = new Thread(  
        new PrintChar('c', 40));  
    thread4.start();  
    try {  
        for (int i = 1; i <= lastNum; i++) {  
            System.out.print(" " + i);  
            if (i == 50) thread4.join();  
        }  
    }  
    catch (InterruptedException ex) {  
    }  
}
```



The numbers after 50 are printed after thread printA is finished.

Thread States

A thread can be in one of five states: New, Ready, Running, Blocked, or Finished.



Thread methods

- **isAlive()**
 - method used to find out the state of a thread.
 - returns true: thread is in the Ready, Blocked, or Running state
 - returns false: thread is new and has not started or if it is finished.
- **interrupt()**
 - If a thread is currently in the Ready or Running state, its interrupted flag is set; if a thread is currently blocked, it is awakened and enters the Ready state, and an `java.io.InterruptedExceptio`n is thrown.
 - The `isInterrupt()` method tests whether the thread is interrupted.

The deprecated `stop()`, `suspend()`, and `resume()` Methods

NOTE:

The Thread class also contains the `stop()`, `suspend()`, and `resume()` methods.

As of Java 2, these methods are *deprecated* (or *outdated*) because they are known to be inherently unsafe.

You should assign `null` to a Thread variable to indicate that it is stopped rather than use the `stop()` method.

Thread Priority

- Each thread is assigned a default priority of `Thread.NORM_PRIORITY` (constant of 5)
- You can reset the priority using `setPriority(int priority)`.
- Some constants for priorities include
`Thread.MIN_PRIORITY`
`Thread.MAX_PRIORITY`
`Thread.NORM_PRIORITY`
- By default, a thread has the priority level of the thread that created it

Thread Scheduling

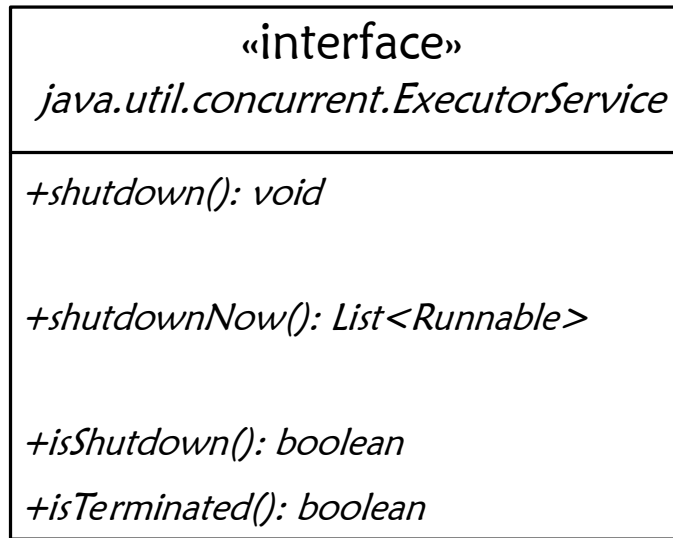
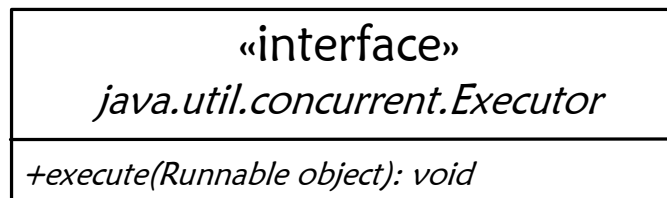
- An operating system's thread scheduler determines which thread runs next.
- Most operating systems use *timeslicing* for threads of equal priority.
- *Preemptive scheduling*: when a thread of higher priority enters the running state, it preempts the current thread.
- *Starvation*: Higher-priority threads can postpone (possibly forever) the execution of lower-priority threads.

Thread Pools

- You learned how to create a thread to run a task like this:
Yield b = new Yield();
Thread t1 = new Thread(b);
t1.start();
or
Thread b = new Yield();
new Thread(b).start();
- This approach is convenient for a single task execution.
- But it is **not efficient for a large number of tasks**, because you have to create a thread for each task
- Starting a new thread for each task could limit throughput and cause poor performance.
- A thread pool is ideal to manage the number of tasks executing concurrently.
- Java provides the **Executor** interface for executing tasks in a thread pool and the **ExecutorService** interface for managing and controlling tasks.

Thread Pools...

- Executor interface for executing Runnable objects in a thread Pool
- **ExecutorService** is subinterface of **Executor**



Executes the runnable task.

- Shuts down the executor, but allows the tasks in the executor to complete. Once shutdown, it cannot accept new tasks.
- Shuts down the executor immediately even though there are unfinished threads in the pool. Returns a list of unfinished tasks.
- Returns true if the executor has been shutdown.
- Returns true if all tasks in the pool are terminated.

Creating Executors

To create an Executor object, use the static **newFixedThreadPool(int)**. methods in the Executors class.

If a thread completes executing a task, it can be reused to execute another task

java.util.concurrent.Executors

+newFixedThreadPool(numberOfThreads: int): ExecutorService

+newCachedThreadPool():
ExecutorService

■ Creates a thread pool with a fixed number of threads executing concurrently. A thread may be reused to execute another task after its current task is finished.

■ Creates a thread pool that creates new threads as needed, but will reuse previously constructed threads when they are available.

Thread Synchronization

- When multiple threads share an object and that object is modified by one or more of the threads, **indeterminate** results may occur. (A shared resource may be corrupted if it is accessed simultaneously by multiple threads.)
- If one thread is in the process of updating a shared object and another thread also tries to update it, it is unclear which thread's update takes effect.
- This can be solved by giving only one thread at a time *exclusive access to code* that manipulates the shared object.
- **Thread synchronization** coordinates access to shared data by multiple concurrent threads.

Thread Synchronization...

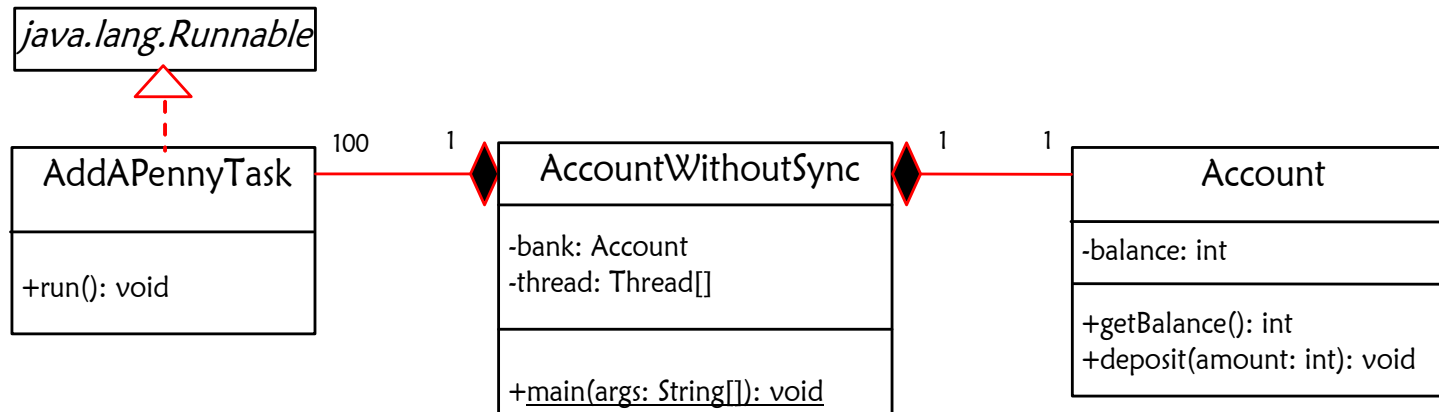
Example: two unsynchronized threads accessing the same bank account may cause conflict.

Step	balance	thread[i]	thread[j]
1	0	<code>newBalance = bank.getBalance() + 1;</code>	
2	0		<code>newBalance = bank.getBalance() + 1;</code>
3	1	<code>bank.setBalance(newBalance);</code>	
4	1		<code>bank.setBalance(newBalance);</code>

- In Step 1, Task 1 gets the balance from the account
- In Step 2, Task 2 gets the same balance from the account
- In Step 3, Task 1 writes a new balance to the account
- In Step 4, Task 2 writes a new balance to the account

Example: Showing Resource Conflict

- **Objective:** Write a program that demonstrates the problem of resource conflict.
- Suppose that you create and launch one hundred threads, each of which adds a penny to an account. Assume that the account is initially empty.



```
Command Prompt
C:\book>java AccountWithoutSync
What is balance ? 5

C:\book>java AccountWithoutSync
What is balance ? 4

C:\book>java AccountWithoutSync
What is balance ? 7

C:\book>
```

[AccountWithoutSync](#)

Run

Synchronizing Object Access

- To solve problems such as the one just seen, use a *lock object*
- **Lock object:** used to control threads that manipulate shared resources
- In Java: Lock interface and several classes that implement it
- a lock object is added to a class whose methods access shared resources, like this:

```
public class BankAccount {  
    private Lock balanceChangeLock;  
    public BankAccount(){  
        balanceChangeLock = new ReentrantLock();  
        ...  
    }  
    ...  
}
```

- Code that manipulates shared resource is surrounded by calls to **lock** and **unlock**:
 - **balanceChangeLock.lock();**
- Manipulate the shared resource
 - **balanceChangeLock.unlock();**
- If code between calls to lock and unlock throws an exception, call to unlock never happens

- To overcome this problem, place call to unlock into a finally clause:

```
public void deposit(double amount) {  
    balanceChangeLock.lock();  
    try {  
        System.out.print("Depositing " + amount);  
        double newBalance = balance + amount;  
        System.out.println(", new balance is " +  
            newBalance); balance = newBalance;  
    } finally {  
        balanceChangeLock.unlock();  
    }  
}
```

- When a thread calls lock, it owns the lock until it calls unlock
- A thread that calls lock while another thread owns the lock is temporarily deactivated
- Thread scheduler periodically reactivates thread so it can try to acquire the lock
- Eventually, waiting thread can acquire the lock

Condition Objects

- To overcome problem, use a condition object
- Condition objects allow a thread to temporarily release a lock, and to regain the lock at a later time
- Each condition object belongs to a specific lock object
- You obtain a condition object with newCondition method of Lock interface:

```
public class BankAccount {  
    public BankAccount(){  
        balanceChangeLock = new ReentrantLock();  
        sufficientFundsCondition =  
            balanceChangeLock.newCondition();  
    ...}
```

...

```
private Lock balanceChangeLock;  
private Condition sufficientFundsCondition;  
}
```

- It is customary to give the condition object a name that describes condition to test
- You need to implement an appropriate test
- As long as test is not fulfilled, call await on the condition object:

```
public void withdraw(double amount){  
    balanceChangeLock.lock();  
    try {  
        while (balance < amount)  
            sufficientFundsCondition.await();  
        ...  
    }finally{  
        balanceChangeLock.unlock(); }  
}
```

- Calling **await**
 - *Makes current thread wait*
 - *Allows another thread to acquire the lock object*
- To unblock, another thread must execute **signalAll** on the same condition object :
 - **sufficientFundsCondition.signalAll();**
- **signalAll** unblocks all threads waiting on the condition
- **signal**: randomly picks just one thread waiting on the object and unblocks it
- **signal** can be more efficient, but you need to know that every waiting thread can proceed
- Recommendation: always call **signalAll**

BankAccountThreadRunner.java

/** This program runs threads that deposit and withdraw money from the same bank account.*/

```
public class BankAccountThreadRunner {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        final double AMOUNT = 100;
        final int REPETITIONS = 100;
        final int THREADS = 100;
        for (int i = 1; i <= THREADS; i++){
            DepositRunnable d = new DepositRunnable(
                account, AMOUNT, REPETITIONS);
            WithdrawRunnable w = new WithdrawRunnable(
                account, AMOUNT, REPETITIONS);
            Thread dt = new Thread(d);
            Thread wt = new Thread(w);
            dt.start();
            wt.start();
        }
    }
}
```

BankAccount.java

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
/** A bank account has a balance that can be changed by
    deposits and withdrawals.*/
public class BankAccount {
    private double balance;
    private Lock balanceChangeLock;
    private Condition sufficientFundsCondition;
    public BankAccount(){
        balance = 0;
        balanceChangeLock = new ReentrantLock();
        sufficientFundsCondition =
            balanceChangeLock.newCondition();
    }
```

```
public void deposit(double amount){
    balanceChangeLock.lock();
    try {
        System.out.print("Depositing " + amount);
        double newBalance = balance + amount;
        System.out.println(", new balance is "+newBalance);
        balance = newBalance;
        sufficientFundsCondition.signalAll();
    }
    finally {
        balanceChangeLock.unlock();
    }
}
```

```
public void withdraw(double amount) throws
    InterruptedException {
    balanceChangeLock.lock();
    try {
        while (balance < amount)
            sufficientFundsCondition.await();
        System.out.print("Withdrawing " + amount);
        double newBalance = balance - amount;
        System.out.println(", new balance is " + newBalance);
        balance = newBalance;
    }
    finally {
        balanceChangeLock.unlock();
    }
}

public double getBalance() {
    return balance;
}
}
```

- Program Run:

- Depositing 100.0, new balance is 100.0
- Withdrawing 100.0, new balance is 0.0
- Depositing 100.0, new balance is 100.0
- Depositing 100.0, new balance is 200.0

...

- Withdrawing 100.0, new balance is 100.0
- Depositing 100.0, new balance is 200.0
- Withdrawing 100.0, new balance is 100.0
- Withdrawing 100.0, new balance is 0.0

- What is the essential difference between calling **sleep** and **await**?
 - A sleeping thread is reactivated when the sleep delay has passed. A waiting thread is only reactivated if another thread has called **signalAll** or **signal**.

Race Conditions

- When threads share a common object, they can conflict with each other
- Sample program: multiple threads manipulate a bank account
- Here is the run method of **DepositRunnable**:

```
public void run(){
    try{
        for (int i = 1; i <= count; i++){
            account.deposit(amount);
            Thread.sleep(DELAY);
        }
    }
    catch (InterruptedException exception){}
}
```

- The **WithdrawRunnable** class is similar

Example

- Create a BankAccount object
- Create two sets of threads:
 - *Each thread in the first set repeatedly deposits \$100*
 - *Each thread in the second set repeatedly withdraws \$100*
- Deposit and withdraw have been modified to print messages:

```
public void deposit(double amount)
{
    System.out.print("Depositing " + amount);
    double newBalance = balance + amount;
    System.out.println(", new balance is "
                                + newBalance);
    balance = newBalance;
}
```

- The result should be zero, but sometimes it is not
- Normally, the program output looks somewhat like this:

Depositing 100.0, new balance is 100.0

Withdrawing 100.0, new balance is 0.0

Depositing 100.0, new balance is 100.0

Depositing 100.0, new balance is 200.0

Withdrawing 100.0, new balance is 100.0

...

Withdrawing 100.0, new balance is 0.0

- But sometimes you may notice messed-up output, like this:

Depositing 100.0Withdrawing 100.0, new balance is 100.0, new balance is -100.0

Scenario to Explain Non-zero Result: Race Condition

1. A deposit thread executes the lines:

```
System.out.print("Depositing " + amount);
```

```
double newBalance = balance + amount;
```

The **balance** variable is still 0, and the **newBalance** local variable is 100

2. The deposit thread reaches the end of its time slice and a withdraw thread gains control

2. The withdraw thread calls the withdraw method which withdraws \$100 from the balance variable; it is now -100

2. The withdraw thread goes to sleep

5. The deposit thread regains control and picks up where it left off; it executes:

```
System.out.println(", new balance is " + newBalance);  
balance = newBalance;
```

- The balance is now 100 instead of 0 because the deposit method used the OLD balance

Race Condition

- Occurs if the effect of multiple threads on shared data depends on the order in which they are scheduled
- It is possible for a thread to reach the end of its time slice in the middle of a statement
- It may evaluate the right-hand side of an equation but not be able to store the result until its next turn:

```
public void deposit(double amount){  
    balance = balance + amount;  
    System.out.print("Depositing " + amount  
        + ", new balance is " + balance);  
}
```

- Race condition can still occur:

balance = the right-hand-side value

BankAccountThreadRunner.java

/ This program runs threads that deposit and withdraw money from the same bank account.*/**

```
public class BankAccountThreadRunner {  
    public static void main(String[] args){  
        BankAccount account = new BankAccount();  
        final double AMOUNT = 100;  
        final int REPETITIONS = 100;  
        final int THREADS = 100;  
        for (int i = 1; i <= THREADS; i++){  
            DepositRunnable d = new DepositRunnable(account,  
AMOUNT, REPETITIONS);  
            WithdrawRunnable w = new WithdrawRunnable(account,  
AMOUNT, REPETITIONS);  
            Thread dt = new Thread(d);  
            Thread wt = new Thread(w);  
            dt.start();  
            wt.start();  
        }  
    }  
}
```

DepositRunnable.java

```
/** A deposit runnable makes periodic deposits to a bank
    account.*/
public class DepositRunnable implements Runnable {
    private static final int DELAY = 1;
    private BankAccount account;
    private double amount;
    private int count;
    public DepositRunnable(BankAccount anAccount,
        double anAmount,int aCount){
        account = anAccount;
        amount = anAmount;
        count = aCount;
    }
```

```
public void run(){  
    try {  
        for (int i = 1; i <= count; i++){  
            account.deposit(amount);  
            Thread.sleep(Delay);  
        }  
    }  
    catch (InterruptedException exception) {}  
}
```

WithdrawRunnable.java

```
/**A withdraw runnable makes periodic withdrawals  
from a bank account.*/  
public class WithdrawRunnable implements Runnable {  
    private static final int DELAY = 1;  
    private BankAccount account;  
    private double amount;  
    private int count;  
    public WithdrawRunnable(BankAccount anAccount,  
        double anAmount,int aCount){  
        account = anAccount;  
        amount = anAmount;  
        count = aCount;  
    }  
}
```

```
public void run(){  
    try{  
        for (int i = 1; i <= count; i++){  
            account.withdraw(amount);  
            Thread.sleep(DELAY);  
        }  
    }  
    catch (InterruptedException exception) {}  
}  
}
```

BankAccount.java

```
/**A bank account has a balance that can be changed by
    deposits and withdrawals.*/
public class BankAccount{
    private double balance;
    public BankAccount(){
        balance = 0;
    }
    public void deposit(double amount){
        System.out.print("Depositing " + amount);
        double newBalance = balance + amount;
        System.out.println(", new balance is " + newBalance);
        balance = newBalance;
    }
}
```

```
public void withdraw(double amount){  
    System.out.print("Withdrawing " +  
amount);  
    double newBalance = balance - amount;  
    System.out.println(", new balance is " +  
newBalance);  
    balance = newBalance;  
}  
public double getBalance(){  
    return balance;  
}  
}
```

Program Run:

Depositing 100.0, new balance is 100.0

Withdrawing 100.0, new balance is 0.0

Depositing 100.0, new balance is 100.0

Withdrawing 100.0, new balance is 0.0

...

Withdrawing 100.0, new balance is 400.0

Depositing 100.0, new balance is 500.0

Withdrawing 100.0, new balance is 400.0

Withdrawing 100.0, new balance is 300.0

synchronized

- Problem: race conditions
- Solution: give exclusive access to one thread at a time to code that manipulates a shared object.
- Synchronization keeps other threads waiting until the object is available.
- The synchronized keyword synchronizes the method so that only one thread can access the method at a time.
- One way to correct the problem make Account thread-safe by adding the synchronized keyword in deposit:

```
public synchronized void deposit(double amount)
```

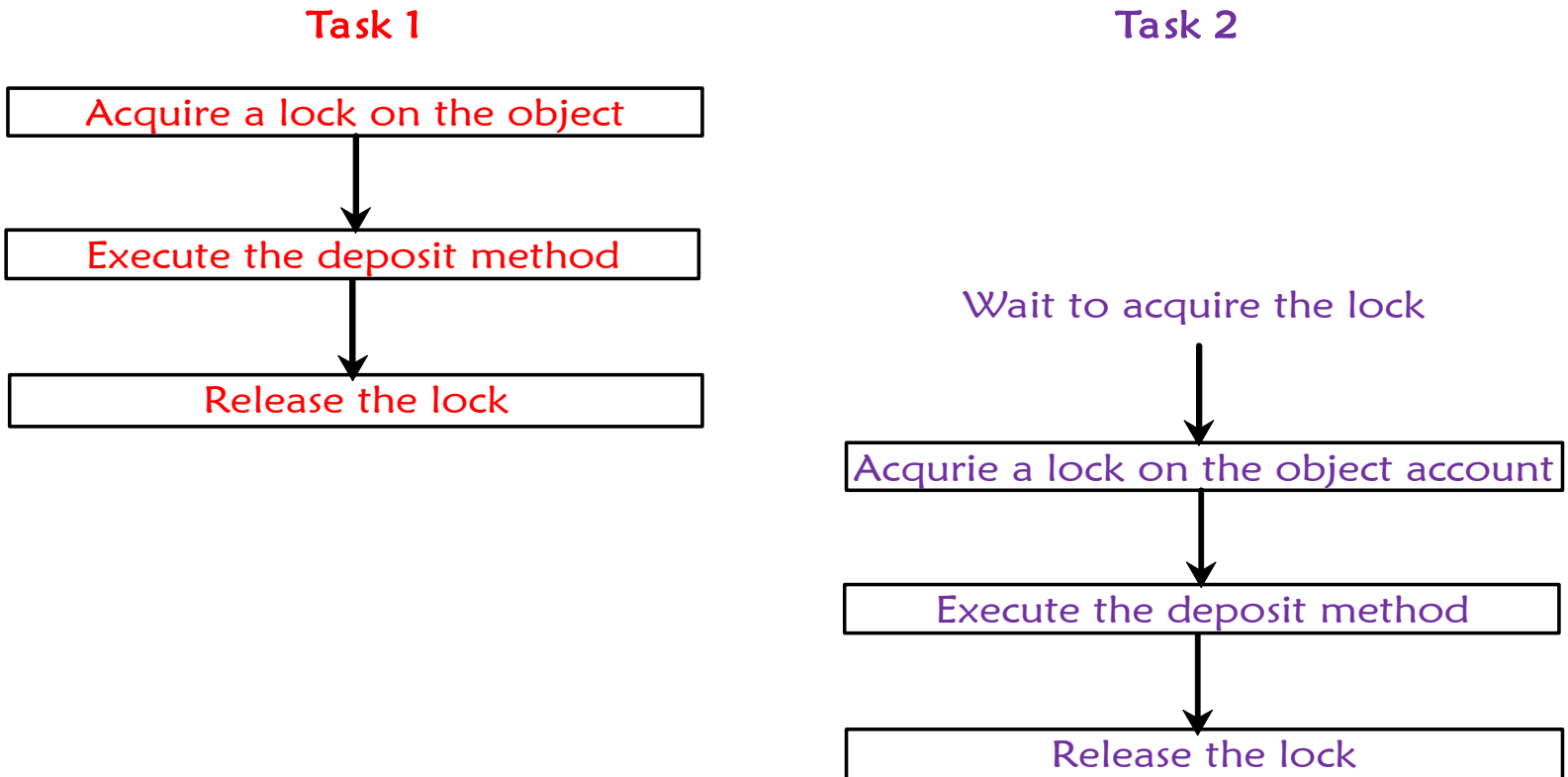
Synchronizing Instance Methods and Static Methods

- A synchronized method acquires a lock before it executes.
- Instance method: the lock is on the object for which it was invoked.
- Static method: the lock is on the class.
- If one thread invokes a synchronized instance method (respectively, static method) on an object, the lock of that object (respectively, class) is acquired, then the method is executed, and finally the lock is released.
- Another thread invoking the same method of that object (respectively, class) is blocked until the lock is released.

Synchronizing Instance Methods and Static Methods

With the deposit method synchronized, the preceding scenario cannot happen.

If Task 2 starts to enter the method, and Task 1 is already in the method, Task 2 is blocked until Task 1 finishes the method.



Synchronizing Statements

- Invoking a synchronized instance method of an object acquires a lock on the object.
- Invoking a synchronized static method of a class acquires a lock on the class.
- A *synchronized block* can be used to acquire a lock on any object, not just *this* object, when executing a block of code.

```
synchronized (expr)  
{ statements; }
```

- **expr** must evaluate to an object reference.
- If the object is already locked by another thread, the thread is blocked until the lock is released.
- When a lock is obtained on the object, the statements in the synchronized block are executed, and then the lock is released.

Synchronizing Statements vs. Methods

- Any synchronized instance method can be converted into a synchronized statement. Suppose that the following is a synchronized instance method:

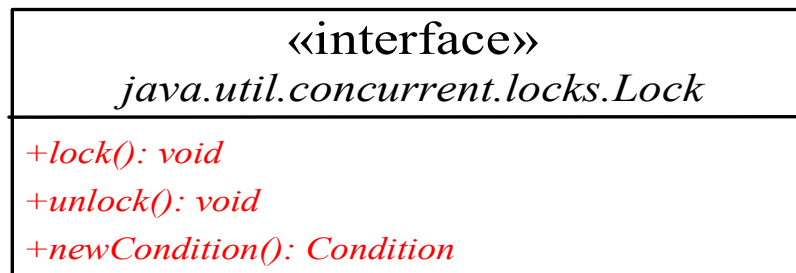
```
public synchronized void xMethod() {  
    // method body  
}
```

- This method is equivalent to

```
public void xMethod() {  
    synchronized (this) {  
        // method body  
    }  
}
```

Synchronization Using Locks

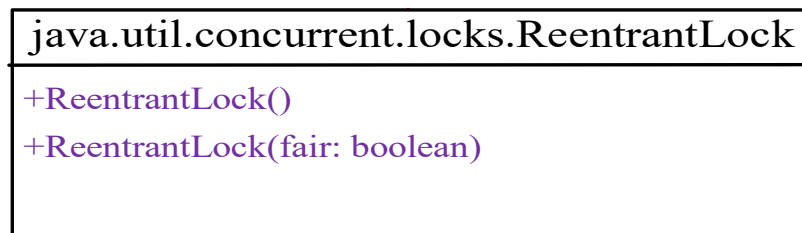
- A synchronized instance method implicitly acquires a lock on the instance before it executes the method.
- You can use locks explicitly to obtain more control for coordinating threads.
- A lock is an instance of the Lock interface, which declares the methods for acquiring and releasing locks.
- newCondition() method creates Condition objects, which can be used for thread communication.



Acquires the lock.

Releases the lock.

Returns a new Condition instance that is bound to this Lock instance.



Same as ReentrantLock(false).

Creates a lock with the given fairness policy. When the fairness is true, the longest-waiting thread will get the lock. Otherwise, there is no particular access order.

Fairness Policy

- **ReentrantLock**: concrete implementation of **Lock** for creating mutually exclusive locks.
- Create a lock with the specified fairness policy.
- **True fairness** policies guarantee the longest-wait thread to obtain the lock first.
- **False fairness** policies grant a lock to a waiting thread without any access order.
- Programs using fair locks accessed by many threads may have poor overall performance than those using the default setting, but have smaller variances in times to obtain locks and guarantee lack of starvation.

Cooperation Among Threads

- Conditions can be used for communication among threads.
- A thread can specify what to do under a certain condition.
- newCondition() method of Lock object.
- Condition methods:
 - await() current thread waits until the condition is signaled
 - signal() wakes up a waiting thread
 - signalAll() wakes all waiting threads

«interface»

java.util.concurrent.Condition

+await(): void

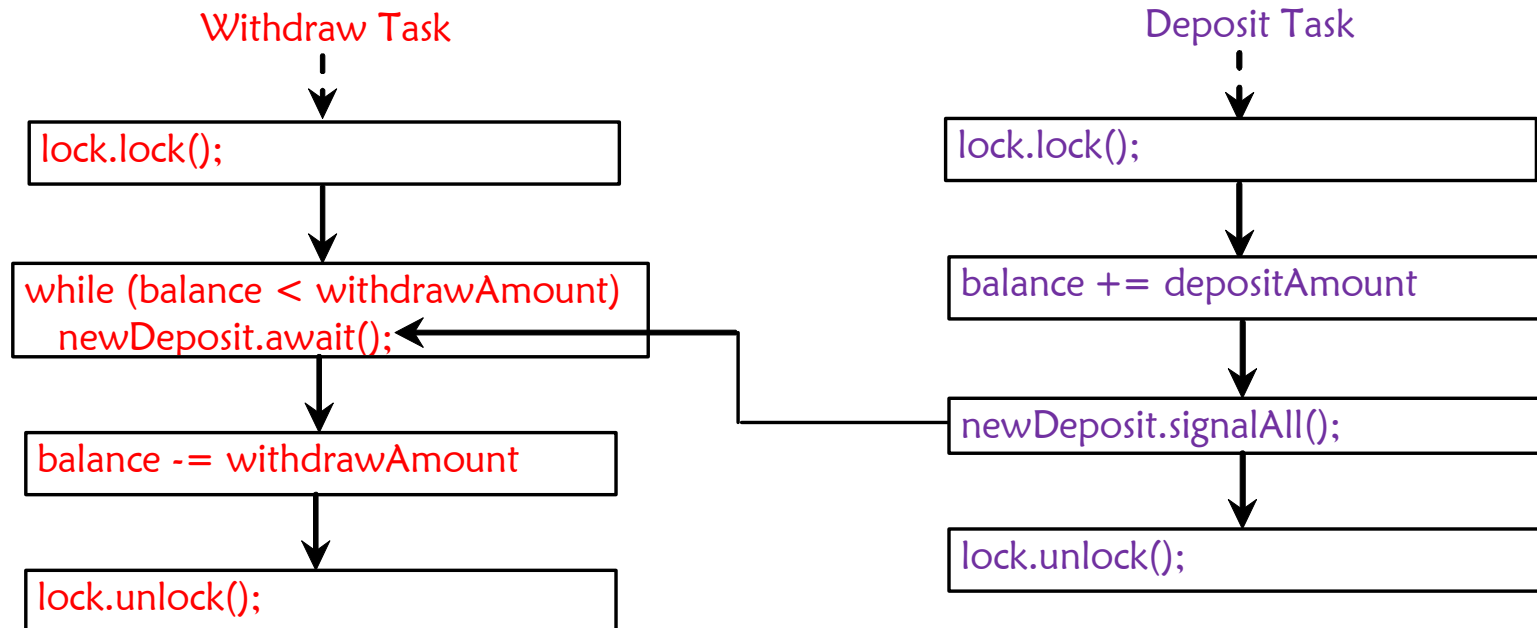
+signal(): void

+signalAll(): Condition

- Causes the current thread to wait until the condition is signaled.
- Wakes up one waiting thread.
- Wakes up all waiting threads.

Cooperation Among Threads

- Lock with a condition to synchronize operations: newDeposit
- If the balance is less than the amount to be withdrawn, the withdraw task will wait for the newDeposit condition.
- When the deposit task adds money to the account, the task signals the waiting withdraw task to try again.
- Interaction between the two tasks:



wait(), notify(), and notifyAll()

- These methods facilitate communication among threads.
- These methods must be called in a synchronized method or a synchronized block on the calling object of these methods.
- Otherwise, an IllegalMonitorStateException would occur.
- The wait() method
 - lets the thread wait until some condition occurs.
- When it occurs, you can use the notify() or notifyAll() methods to notify the waiting threads to resume normal execution.
- The notifyAll() method wakes up all waiting threads, while notify() picks up only one thread from a waiting queue.

Example: Using Monitor

Task 1

```
synchronized (anObject) {  
    try {  
        // Wait for the condition to become true  
        while (!condition)  
            anObject.wait();  
  
        // Do something when condition is true  
    }  
    catch (InterruptedException ex) {  
        ex.printStackTrace();  
    }  
}
```

resume

Task 2

```
synchronized (anObject) {  
    // When condition becomes true  
    anObject.notify(); or anObject.notifyAll();  
    ...  
}
```

- When the thread is restarted after being notified, the lock is automatically reacquired.
- The wait(), notify(), and notifyAll() methods on an object are analogous to the await(), signal(), and signalAll() methods on a condition.